

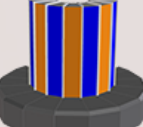
Séance du 18 novembre 2022

Fabrice.Kordon@lip6.fr



Annonce — TME solo

2

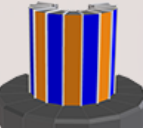
 **2 décembre 2022 8h45-10h30**

 par demi groupe (45mn)

▶ 8h45-9h30

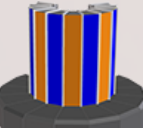
▶ 9h30-10h15

 Votre horaire envoyé par par email avant le 2 décembre

 **Salles**

 14-15/401

 14-15/408

 **la machine et vous...**

 Soumission via Moodle

Un peu de «learning analytics»

Fabrice.Kordon@lip6.fr



Learning Analytics?

4

Analyse de l'apprentissage

-  Analyse de données objectives sur un groupe d'«apprenants»
-  Facilité par l'usage d'infrastructures d'enseignements
 - ▶ Ici, Les vidéos du cours via Youtube Analytics

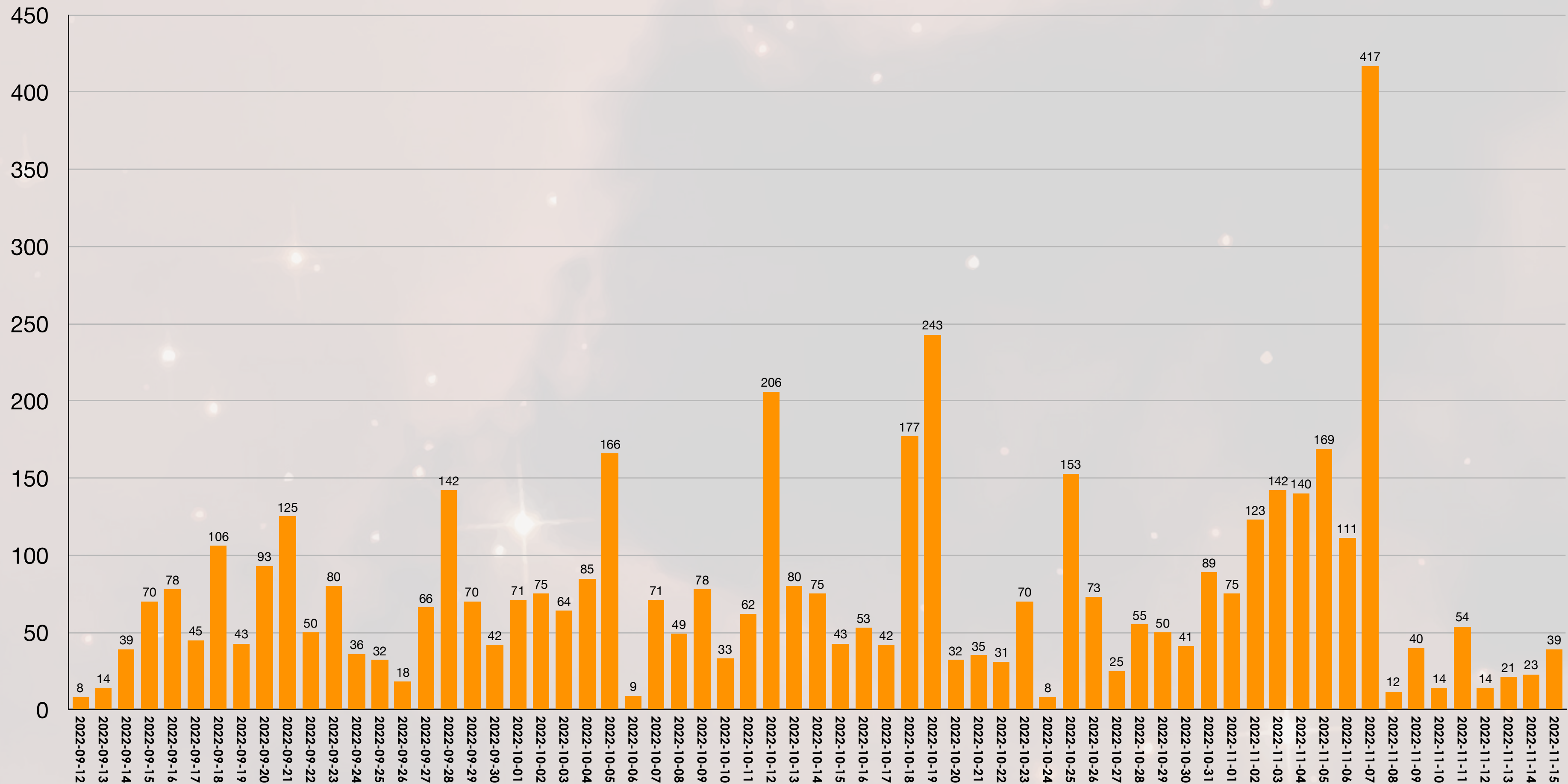
Analysons donc...

-  Nombre de vues...
-  Durée de visionnage
-  Comparaison avec la couverture «attendue»

Quelques chiffres...

Vues quotidiennes

F. Kordon - Sorbonne Université - Creative Commons 2022



du 12 septembre 2022 au 16 novembre 2022

Total vues

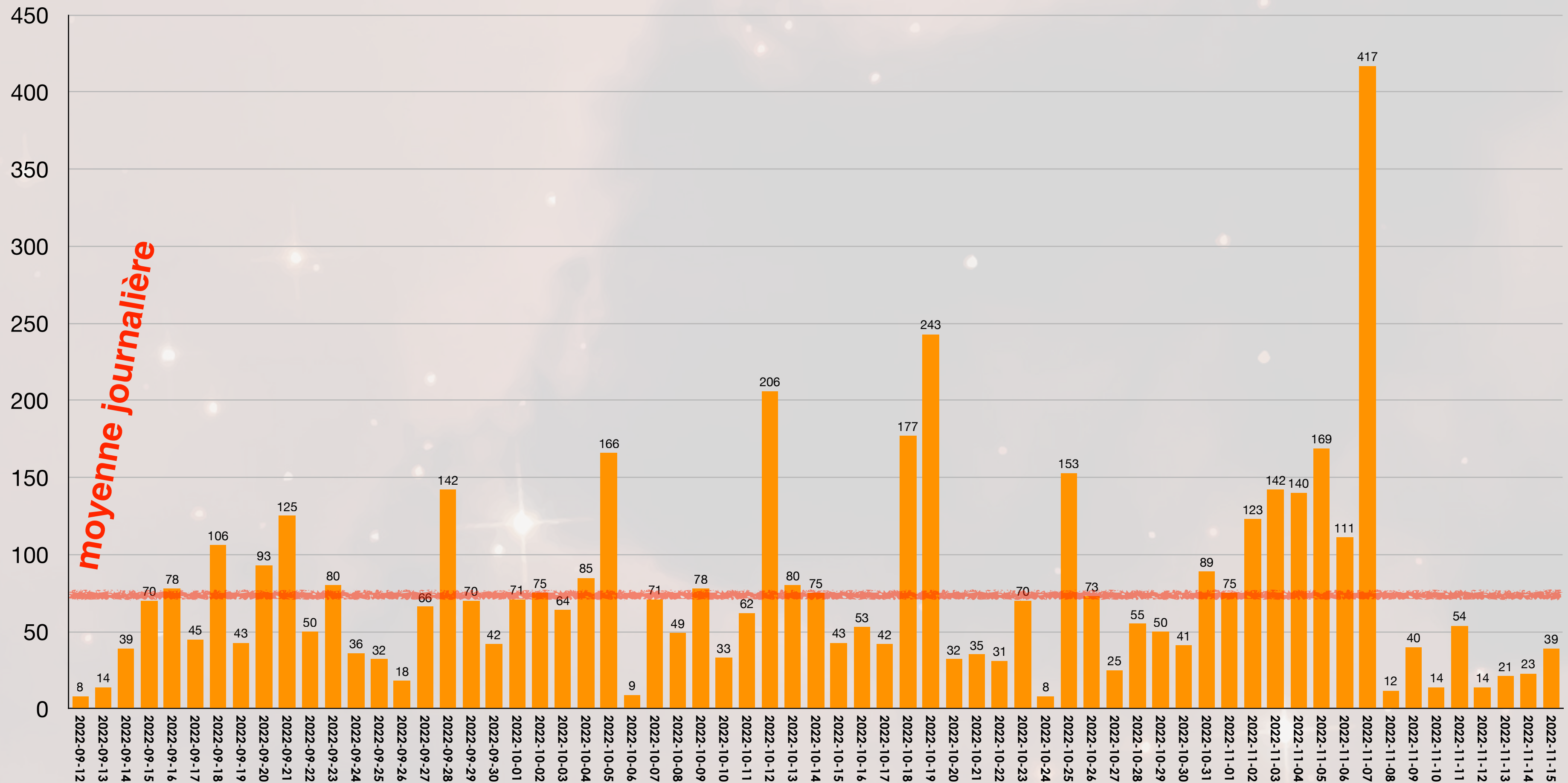
4 988

Quelques chiffres...

5

Vues quotidiennes

F. Kordon - Sorbonne Université - Creative Commons 2022



du 12 septembre 2022 au 16 novembre 2022

Total vues

4 988

Quelques chiffres...

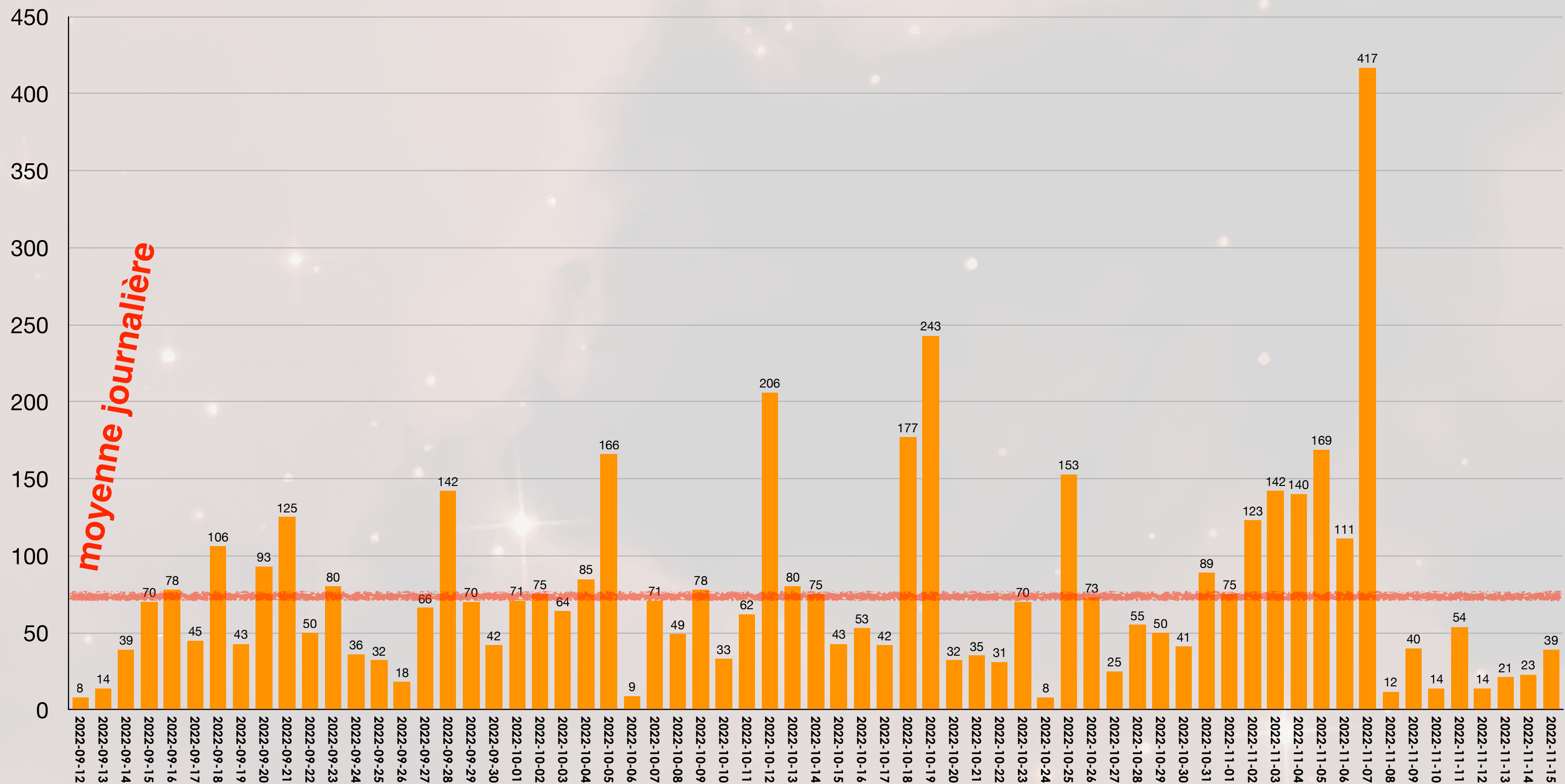
Durée totale

23 742 mn

5

Vues quotidiennes

F. Kordon - Sorbonne Université - Creative Commons 2022

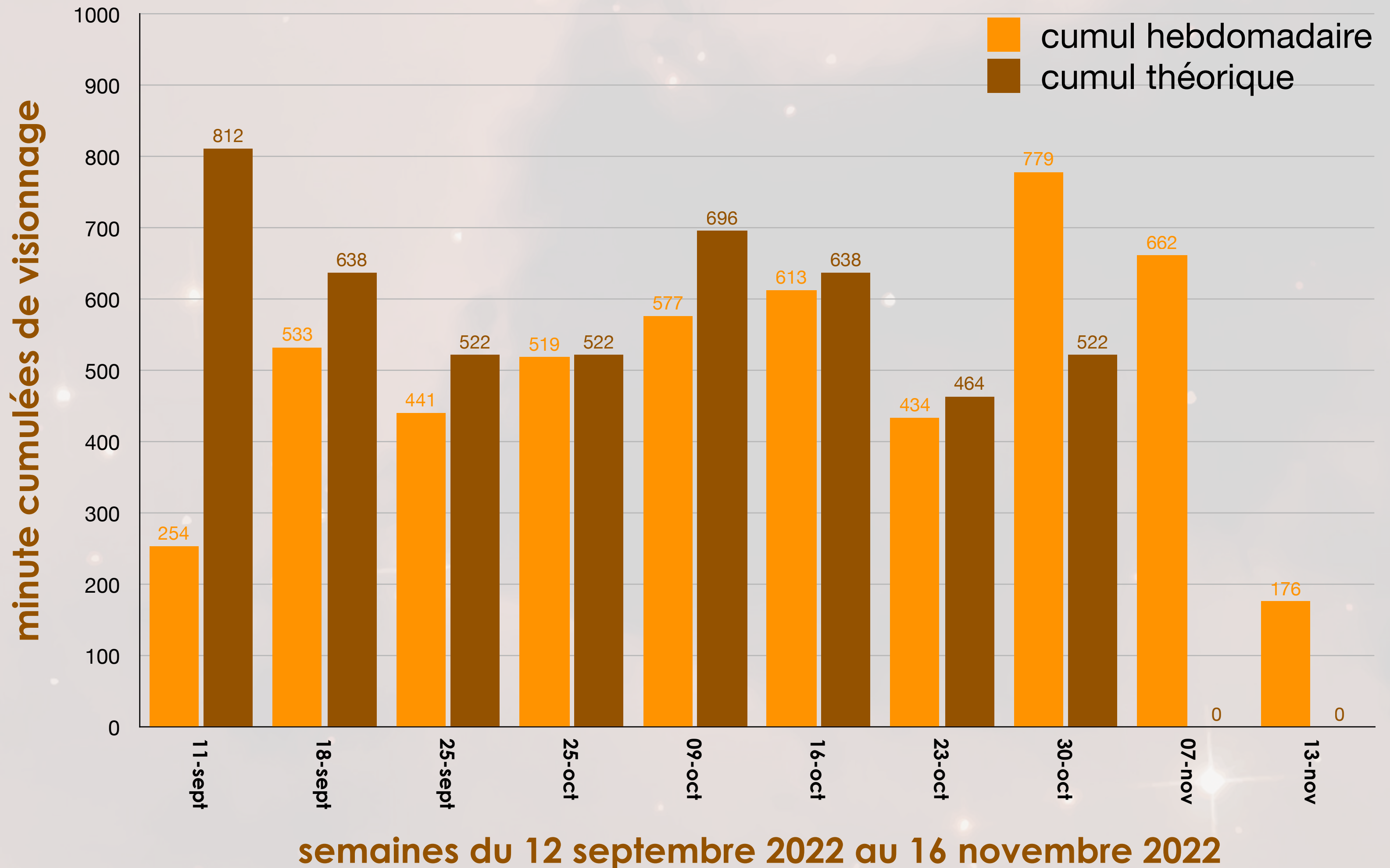


du 12 septembre 2022 au 16 novembre 2022

Durées & durées théoriques par semaine...

6

Cumul hebdomadaire & théorique (vues)



Que signifient ces chiffres?

7

Données à ce jour

-  En ligne : 84 vidéos et 762 minutes (sans les corrigés des partiels)
-  Inscrits : 58 étudiants
-  Attendu:
 - ▶ 44 196 minutes pour tous
 - ▶ 4 872 vues pour tous

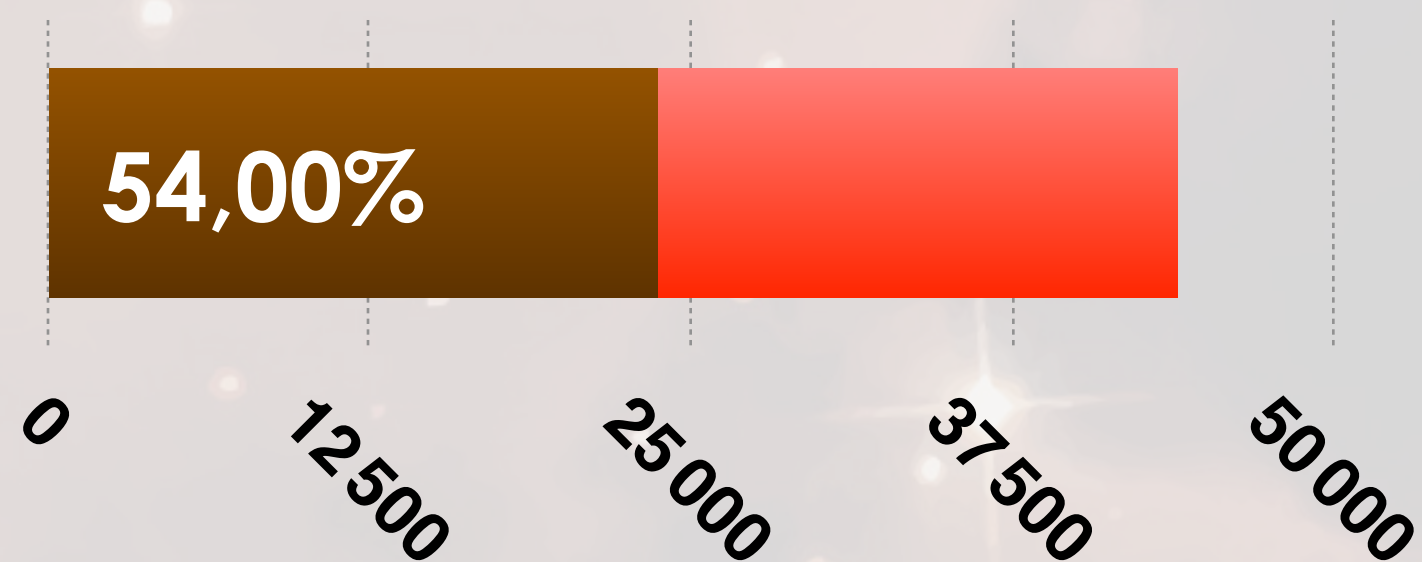
Que signifient ces chiffres?

7

Données à ce jour

-  En ligne : 84 vidéos et 762 minutes (sans les corrigés des partiels)
-  Inscrits : 58 étudiants
-  Attendu:
 - ▶ 44 196 minutes pour tous
 - ▶ 4 872 vues pour tous

Portion du visionnage (mn)



Constatations

- ▶ un peu meilleur que l'an dernier! e.g. +20% en visionnage (avec un effectif similaire)
- ▶ Tout le monde ne regarde pas tout!
- ▶ Vous ne regardez pas les vidéos «d'une traite»
- ▶ Inconnue, la comptabilisation des vidéos téléchargées...

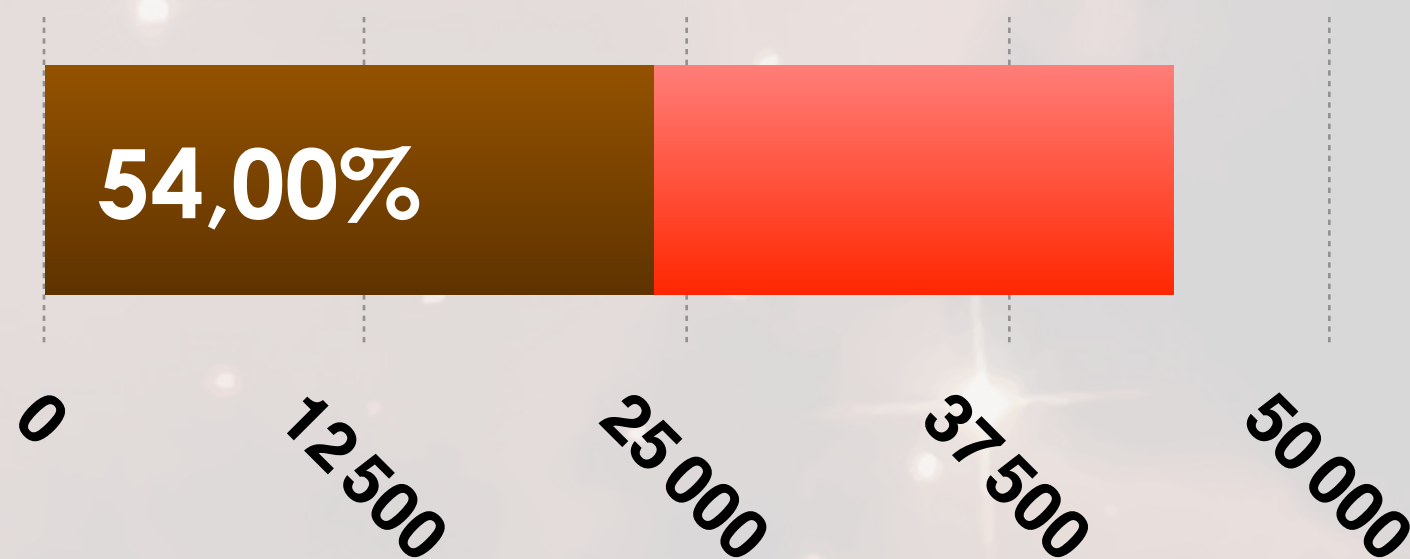
Que signifient ces chiffres?

7

Données à ce jour

-  En ligne : 84 vidéos et 762 minutes (sans les corrigés des partiels)
-  Inscrits : 58 étudiants
-  Attendu:
 - ▶ 44 196 minutes pour tous
 - ▶ 4 872 vues pour tous

Portion du visionnage (mn)



Sans les corrigés des partiels
ils gonflent le taux de visionnage

Constatations

- ▶ un peu meilleur que l'an dernier! e.g. +20% en visionnage (avec un effectif similaire)
- ▶ Tout le monde ne regarde pas tout!
- ▶ Vous ne regardez pas les vidéos «d'une traite»
- ▶ Inconnue, la comptabilisation des vidéos téléchargées...

À propos du partiel...

Fabrice.Kordon@lip6.fr



Éléments sur le partiel

9

Participation

 55 copies collectées sur 58 inscrits à l'UE $\Rightarrow$ **94,82%**

 25 copies corrigées $\Rightarrow$ **46,29%**

► **Barème corrigé sur 20 points (barème en cours d'étude)!**

Éléments sur le partiel

9

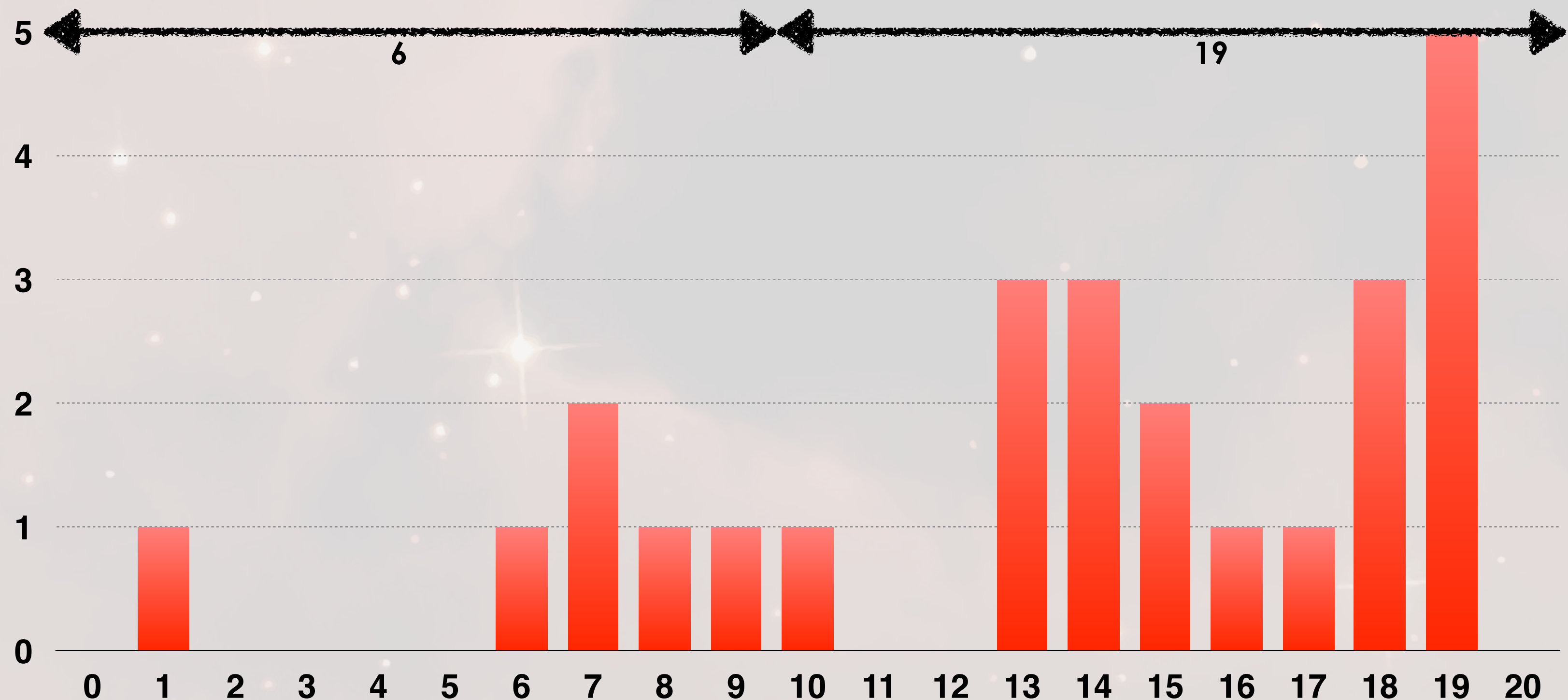
Participation

 55 copies collectées sur 58 inscrits à l'UE → **94,82%**

 25 copies corrigées → **46,29%**

► **Barème corrigé sur 20 points (barème en cours d'étude)!**

Distribution des notes (sur 18 copies)



Éléments sur le partiel

9

Participation

 55 copies collectées sur 58 inscrits à l'UE $\Rightarrow$ **94,82%**

 25 copies corrigées $\Rightarrow$ **46,29%**

► Barème corrigé sur 20 points (barème en cours d'étude)!

Distribution des notes (sur 18 copies)



Bien disposer le correcteur à votre égard...

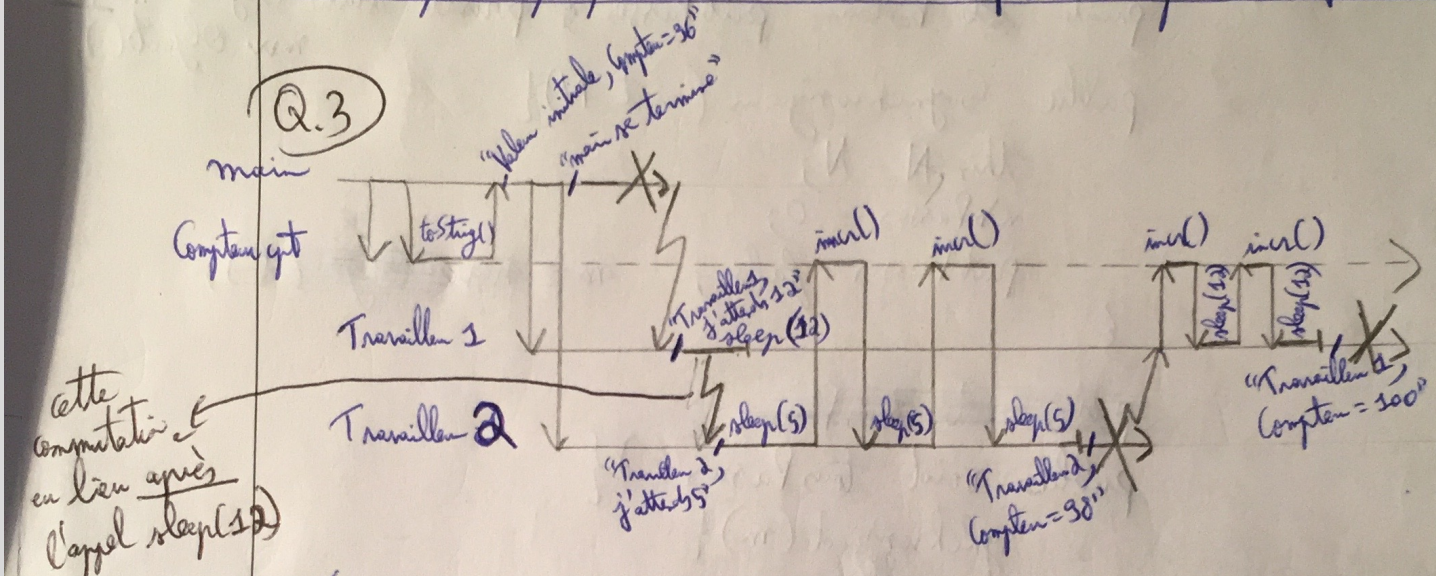
10

```
private int n;
private int nbThreads = 0;
private int nbThreads = 0;
private ReentrantLock lock = new ReentrantLock();
private Condition attenteFinRound = new Condition(lock);
```

```
3) * public void transParer() {
    nbThreads++;
    if (nbThreads < n) {
        lock.lock();
        try { nbThreads++;
            if (nbThreads < n) {
                attenteFinRound.await();
            } else {
                nbThreads = 0;
                attenteFinRound.signalAll();
            }
        } finally {
            lock.unlock();
        }
    }
}
```

```
* Synchroniseur(int taille) {
    this.n = taille;
}
```

Q.2 Si on enlève "static" de la ligne 13, l'objet ~~sur lequel on se synchronise~~ devient privé à chaque instance (On aura une copie pour chaque instance) et donc ~~il y~~ aura plus de protection en parallèle (on devient un attribut d'instance et chaque instance se synchronise sur son attribut (sa copie) propre à elle donc pas de protection concurrente).



↔ signifie commutation et ↑↓ signifie un appel à une méthode.
Le compteur qui n'est pas une file d'exécution mais représente la ressource critique manipulée.

Exercice 3:

Q.1 Source manquante)

```
//attente de la fin du tri
for (int i = 0; i <= taille; i++) {
    comp[i].join();
}
```

Q.2 Je propose 3 variables locales: ~~et un objet m~~
~~nbPasses~~ et ~~peutPasser (booléen)~~
~~nbPasses~~ sert à incrémenter au fin et à mesurer le nombre de thread qui ont invoqué la méthode. Et de que le dernier thread arrive, elle incrémente ~~nbPasses~~ par un appel à une méthode et on aura ~~nbPasses = N~~ et donc cette dernière thread mettra ~~nbPasses~~ à ~~taille~~.

Bien disposer le correcteur à votre égard...

10

private
private
~~private~~
private
private

3) * public
~~lock~~
if
lock
try {
nb++
if (a
3 el

3
3 finally

3
* Synchron
thi
3

Ex 1:

Q1:

notify: reveille uniquement une instance de processus qui était en attente.

notifyAll: reveille toutes les instances de processus qui étaient en attente

Q2:

non

le notifyAll reveillera tous les threads endormies sur un wait

ils seront alors tous en concurrence pour accéder à la ressource critique qui est protégée

Q3:

programmation répartie: répartition des tâches sur un seul cœur, en parallèle.

programmation multicœur: utilisation des caractéristiques de l'ordinateur (multicœur) pour exécuter des tâches sur les différents cœurs en vue d'optimiser l'effectivité temporelle de l'exécution (la vitesse).

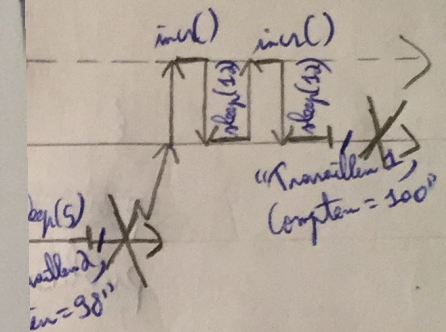
répartition des tâches sur les différents cœurs en parallèle.

Q4:

utilisation du join() dans le programme principal:

```
public class EXOCOURS {
    public static void main (String [] args) {
        try {
            for (int i = 0; i < 5; i++) {
                new Travail().start().join();
            }
        } catch (InterruptedException e) {
            System.out.println("pb!");
        }
        System.out.println("tout est terminé!");
    }
}
```

la ligne 13, l'objet ~~a~~ m
iet privé à chaque instance
tance) et donc ~~sa~~ y
ile (on définit un attribut
recharge son attribut
de protection ~~concurrente~~)



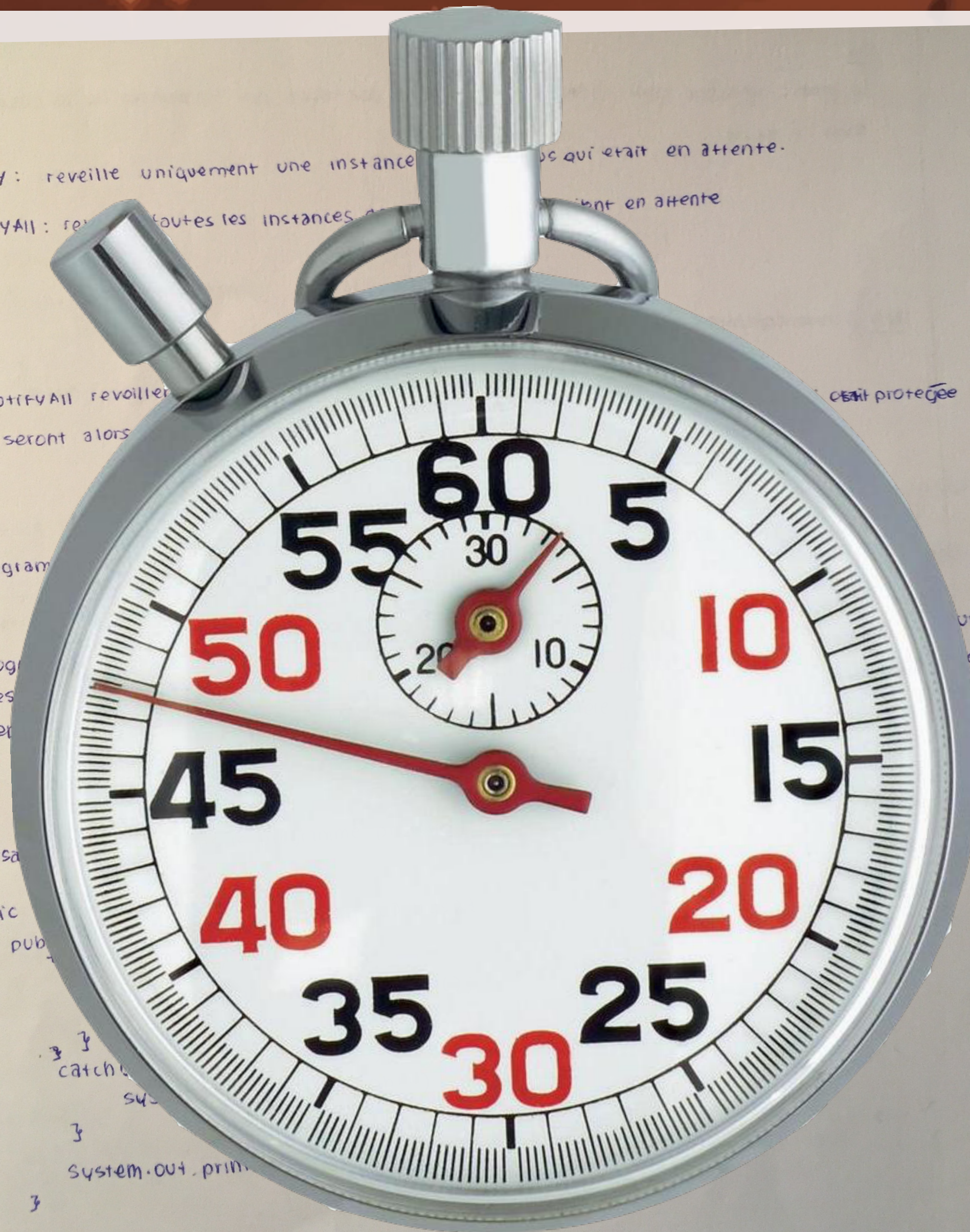
signifie un appel à une méthode.
représente la ressource critique manipulée.

~~une d'être en attente~~
~~concurrente on doit avoir~~
~~et finit~~
~~et le variable~~
~~table~~
~~et le objet~~
~~devenir~~

locaux: et un objet m
peut passer (boolean) ↓
et à mesure ~~de~~ le
la méthode. Et de
le incrémente nbPasses
ne nbPasses = N et
neut passer à t

Bien disposer le correcteur à votre égard...

10



private
private
~~private~~
private
private

Ex1:

Q1:

notify: reveille uniquement une instance qui était en attente.

notifyAll: reveille toutes les instances qui sont en attente

Q2:

non
le notifyAll reveille
ils seront alors

est protégée

Q3:

program

progr
des
rep

Q4:

utilisa

public

pub

}
catch

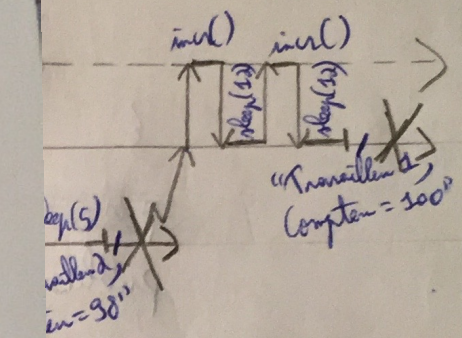
sys

}

System.out.println

}

la ligne 13, l'objet ~~se~~ m
iet privé à chaque instance
tance) et donc ~~se~~ y
ile (on deviet un attribut
recharge son attribut
de protection ~~concurrente~~)



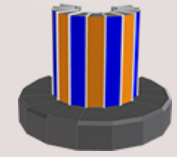
ou exécuter signifie un appel à une méthode.
on la vitesse, on représente la ressource critique
manipulée.

~~une d'être en attente~~
~~concurrente on doit avoir~~
~~Attente~~
~~et le variable~~
~~table et de la~~
~~donnée~~

locaux: et un objet m
peut passer (boolean) ↓
et à mesure ~~de~~ le
la méthode. Et de
le incrémenter nPasses
ne nPasses = N et
neut passer à t

Questions de cours, Q1

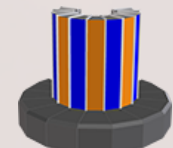
11

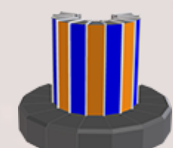


Expliquez brièvement les objectifs spécifiques d'un **ReentrantReadWriteLock**

Questions de cours, Q1

11

 Expliquez brièvement les objectifs spécifiques d'un **ReentrantReadWriteLock**

 Le **ReentrantReadWriteLock** permet de différencier deux types d'accès : un accès en écriture par des “écri- vains” (un seul écrivain peut alors exécuter du code dans le **ReentrantReadWriteLock**) et un accès en lecture par des “lecteurs” (et plusieurs lecteurs peuvent alors exécuter du code dans le **ReentrantReadWriteLock**).

Questions de cours, Q2

12

```
class MaClasse implements Runnable {
    private int monId;

    MaClasse(int id) {
        monId = id;
    }

    public void run() {
        try {
            if (monId % 2 == 1) {
                Thread.sleep(300);
            }
            System.out.println(" " + monId + " démarre");
            Thread.sleep(100);
            System.out.println(" " + monId + " termine");
        }
        catch (Exception e) {
            System.err.println(" " + monId + " probleme avec " + e);
        }
    }
}

public class Chronogramme {
    public static void main(String []args) {
        System.out.println("PP, je me lance");
        for (int i = 1 ; i <= 3 ; i++) {
            MaClasse mc = new MaClasse(i);
            Thread t = new Thread(mc);
            t.start();
        }
        System.out.println("PP, je me termine");
    }
}
```

```
$ java Chronogramme
PP, je me lance
PP, je me termine
  2 démarre
  2 termine
  1 démarre
  3 démarre
  1 termine
  3 termine
$ java Chronogramme
PP, je me lance
PP, je me termine
  2 démarre
  2 termine
  1 démarre
  3 démarre
  3 termine
  1 termine
```


Questions de cours, Q2

12

```
class MaClasse implements Runnable {
    private int monId;

    MaClasse(int id) {
        monId = id;
    }

    public void run() {
        try {
            if (monId % 2 == 1) {
                Thread.sleep(300);
            }
            System.out.println("  " + monId + " démarre");
            Thread.sleep(100);
            System.out.println("  " + monId + " termine");
        }
        catch (Exception e) {
            System.err.println("  " + monId + " probleme avec " + e);
        }
    }
}
```

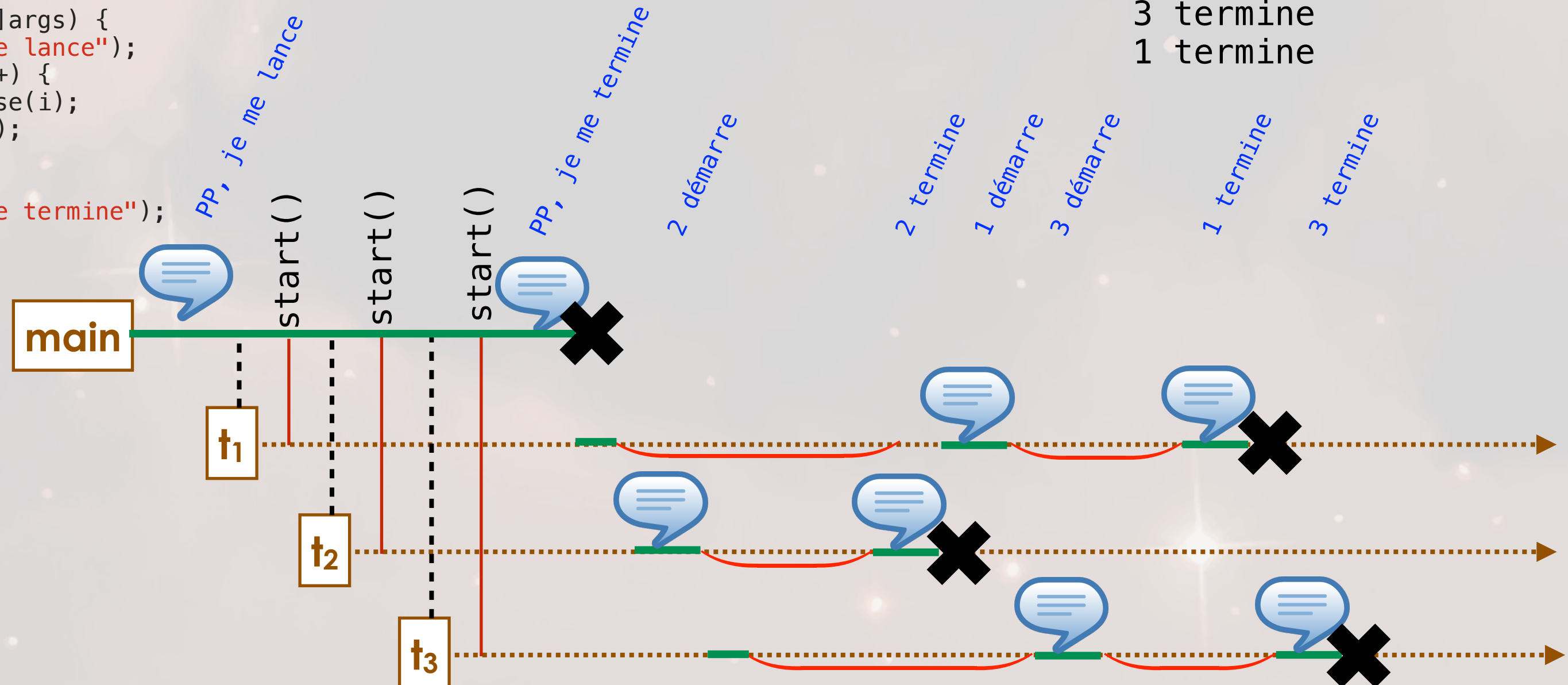
```
public class Chronogramme {
    public static void main(String []args) {
        System.out.println("PP, je me lance");
        for (int i = 1 ; i <= 3 ; i++) {
            MaClasse mc = new MaClasse(i);
            Thread t = new Thread(mc);
            t.start();
        }
        System.out.println("PP, je me termine");
    }
}
```

```
$ java Chronogramme
PP, je me lance
PP, je me termine
```

```
2 démarre
2 termine
1 démarre
3 démarre
1 termine
3 termine
```

```
$ java Chronogramme
PP, je me lance
PP, je me termine
```

```
2 démarre
2 termine
1 démarre
3 démarre
3 termine
1 termine
```



Questions de cours, Q2

12

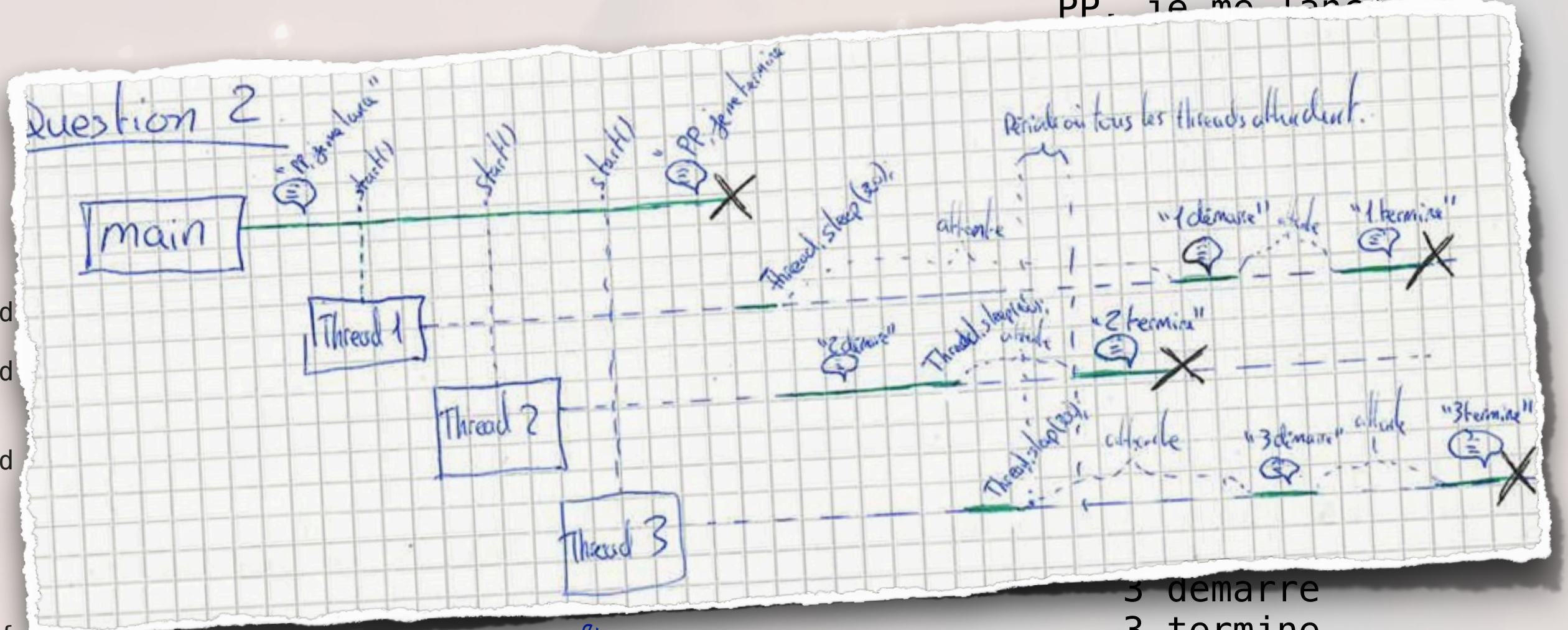
```
class MaClasse implements Runnable {
    private int monId;

    MaClasse(int id) {
        monId = id;
    }

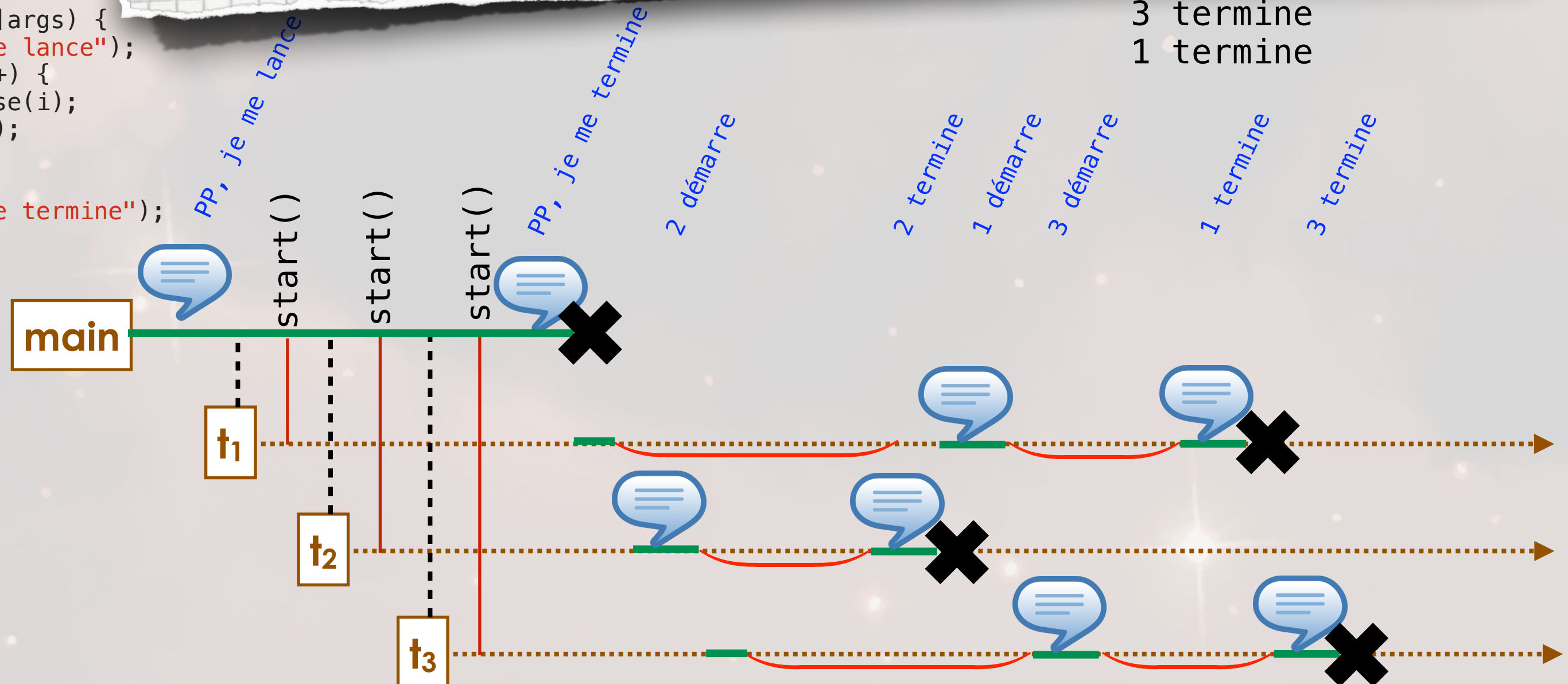
    public void run() {
        try {
            if (monId % 2 == 1) {
                Thread.sleep(300);
            }
            System.out.println("  " + monId);
            Thread.sleep(100);
            System.out.println("  " + monId);
        } catch (Exception e) {
            System.err.println("  " + monId);
        }
    }
}
```

```
public class Chronogramme {
    public static void main(String []largs) {
        System.out.println("PP, je me lance");
        for (int i = 1 ; i <= 3 ; i++) {
            MaClasse mc = new MaClasse(i);
            Thread t = new Thread(mc);
            t.start();
        }
        System.out.println("PP, je me termine");
    }
}
```

\$ java Chronogramme
PP, je me lance

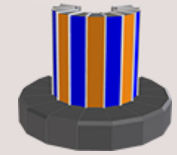


3 démarre
3 termine
1 termine



Questions de cours, Q3

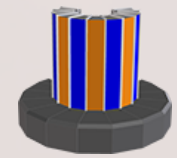
13



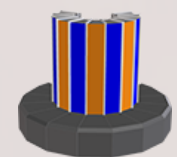
On observe que dans les deux exécutions, “2” démarre et se termine avant “1” et “3”. Cette observation est-elle une généralisation ? justifiez brièvement votre réponse (une réponse non justifiée ne sera pas considérée).

Questions de cours, Q3

13



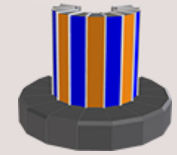
On observe que dans les deux exécutions, “2” démarre et se termine avant “1” et “3”. Cette observation est-elle une généralisation ? justifiez brièvement votre réponse (une réponse non justifiée ne sera pas considérée).



Dans les faits, les commutations sont aléatoires mais le fait que les threads ayant un numéro impair débutent avec 300ms de retard fait que la thread 2 aura toujours le temps de terminer son exécution avant que les autres ne puissent se réveiller de leur premier **sleep(300)**. Attention si plus de 3 threads bien sûr (thread 4 en concurrence avec thread 2).

Un système producteur/consommateur

14

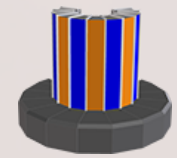


Nous souhaitons élaborer un système producteur/consommateur avec **nbProd** producteurs et **nbCons** consommateurs (**nbProd** et **nbCons** sont des constantes dont les valeurs, indiquées sur la ligne de commande, changent au gré des exécutions). Producteurs et consommateurs communiquent via une “boite” comme indiqué en ci-dessous. Chaque instance (producteur ou consommateur) doit disposer de son propre fil d'exécution.



Un système producteur/consommateur

15

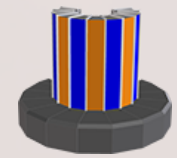


Outre la “classe principale” appelée **ProdCons**, le programme comporte 3 classes différentes :

-  **Prod**, il s'agit de la classe qui implémente le code d'un producteur. Chaque instance de Prod envoie exactement $2 \times \text{nbCons}$ messages avant de se terminer.
-  **Cons**, il s'agit de la classe qui implémente le code d'un consommateur. Toutes les instances de **Cons** doivent traiter le même nombre de messages avant de se terminer.
-  **Boite**, c'est la classe qui implémente la “boite” dans laquelle le producteur produit un message lu par un consommateur. À la terminaison du système, la boîte est vide

Un système producteur/consommateur, Q1

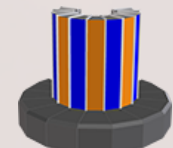
16

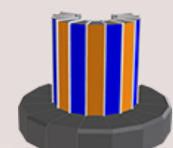


Quelle(s) classes doivent respecter l'interface **Runnable** ? expliquez brièvement pourquoi

Un système producteur/consommateur, Q1

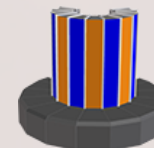
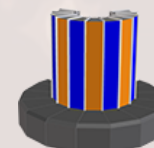
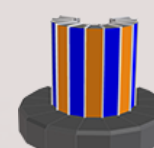
16

 Quelle(s) classes doivent respecter l'interface **Runnable** ? expliquez brièvement pourquoi

 **Prod** et **Cons** sont des éléments actifs dont chaque instance va s'exécuter dans une thread et doivent donc respecter l'interface **Runnable**. **Boite** est un élément passif (qui ne respecte donc pas cette interface) qui sera manipulé en concurrence entre les instances de **Prod** et les instances de **Cons**.

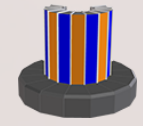
Un système producteur/consommateur

17

-  Les classes **Prod** et **Cons** possèdent deux attributs privés: un identifiant et la référence vers le système de communication qui va permettre aux objets **Prod** de produire des messages consommés par les objets **Cons**.
-  Outre d'autres paramètres que vous déterminerez, **Prod** reçoit dans son constructeur le nombre de consommateurs avec lesquels il devra communiquer. Cette dernière valeur sera stockée dans l'attribut privé **nbCons**.
-  Outre d'autres paramètres que vous déterminerez, **Cons** reçoit dans son constructeur le nombre de message qu'il recevra (ce dernier dépend du nombre de producteurs). Cette dernière valeur sera stockée dans l'attribut privé **maxMsg**.

Un système producteur/consommateur, Q2

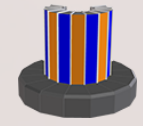
18



Donnez le code du constructeur de **Prod** et de **Cons**. Justifiez brièvement vos choix

Un système producteur/consommateur, Q2

18



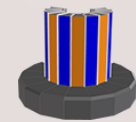
Donnez le code du constructeur de **Prod** et de **Cons**. Justifiez brièvement vos choix

```
class Prod implements Runnable {  
    private int monId;  
    private int nbCons;  
    private Boite mi;  
  
    Prod(int id, Boite m, int nbc) {  
        monId = id;  
        mi = m;  
        nbCons = nbc;  
    }  
}
```

```
class Cons implements Runnable {  
    private int monId;  
    private Boite mi;  
    private int maxMsg = 0;  
  
    Cons(int id, Boite m, int maxMess) {  
        monId = id;  
        mi = m;  
        maxMsg = maxMess ;  
    }  
}
```

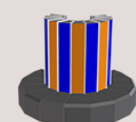

Un système producteur/consommateur, Q2

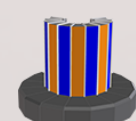
18

 Donnez le code du constructeur de **Prod** et de **Cons**. Justifiez brièvement vos choix

```
class Prod implements Runnable {  
    private int monId;  
    private int nbCons;  
    private Boite mi;  
  
    Prod(int id, Boite m, int nbc) {  
        monId = id;  
        mi = m;  
        nbCons = nbc;  
    }  
}
```

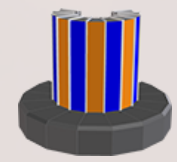
```
class Cons implements Runnable {  
    private int monId;  
    private Boite mi;  
    private int maxMsg = 0;  
  
    Cons(int id, Boite m, int maxMess) {  
        monId = id;  
        mi = m;  
        maxMsg = maxMess ;  
    }  
}
```

 Outre les informations désignées explicitement, Il faut transmettre à l'objet son identifiant (un entier) et une référence vers le système de communication qui est de type UneInteraction. Ici, c'est le programme principal qui donnerait la valeur de l'identifiant aux **Prod** et aux **Cons**.

 L'identifiant est généré automatiquement à partir d'une variable static partagée entre toutes les instances des classes mais à condition que : soit 1) il y a un système de protection contre les accès concurrents, soit 2) qu'une mention explicite au côté séquentiel probable de la "classe principale" soit mentionné.

Un système producteur/consommateur, Q3

19



Écrivez le source de la classe **ProdCons** qui constitue le programme principal du système. Vous pourrez utiliser la méthode statique **parseInt** de la classe Integer qui transforme en int la chaîne de caractères qu'elle prend en paramètre.

Un système producteur/consommateur, Q3

20

```
public class ProdCons {

    static int nbProd;
    static int nbCons;

    public static void main(String []args) {
        try {
            nbProd = Integer.parseInt(args[0]);
            nbCons = Integer.parseInt(args[1]);
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.err.println ("Arguments, nb producteurs, nb consommateurs.");
            return;
        }

        Boite inter = new Boite();

        System.out.println("PP, creation des producteurs");
        for (int i = 1; i <= nbProd ; i++) {
            Prod p = new Prod(i, inter, nbCons);
            Thread t = new Thread (p);
            t.start();
        }
        System.out.println("PP, creation des consommateurs");
        for (int i = 1; i <= nbCons ; i++) {
            new Thread (new Cons(i, inter, 2*nbProd)).start();
        }
        System.out.println("PP, je me termine");
    }
}
```



```
public class ProdCo
```

```
static int nbP
```

```
static int nbC
```

```
public static v
```

```
try {
```

```
nbProd
```

```
nbCons
```

```
} 
```

```
catch (Arra
```

```
System.
```

```
return;
```

```
} 
```

```
Boite inter
```

```
System.out.p
```

```
for (int i =
```

```
Prod p =
```

```
Thread t =
```

```
t.start()
```

```
} 
```

```
System.out.p
```

```
for (int i =
```

```
new Threa
```

```
} 
```

```
System.out.p
```

```
public class ProdCons {
    public static void main (String[] args) {
        int nbProd;
        int nbCons;
    }
}
```

```
try {
```

```
nbProd = Integer.parseInt(args[0]);
```

```
nbCons = Integer.parseInt(args[1]);
```

```
}
```

```
catch (ArrayIndexOutOfBoundsException e) {
```

```
System.out.println("Pas assez de parametre!!!);
```

```
System.exit(-1);
```

```
}
```

```
for (int i = 0; i < nbProd; i++) {
```

```
new Thread(new Prod(i, nbCons), e))
```

```
• start();
```

```
}
```

```
int nbMsg = nbProd / nbCons; // nombre Msg des msg
```

```
for (int j = 0; j < nbCons; j++) {
```

```
new Thread(new Cons(j, nbMsg), e))
```

```
• start();
```

```
}
```

```
S.o.p("Main fini");
```

*faute de
faute absurde !!!*

```
mateurs.");
```



```

3. public class ProdCons {
    public static void main (String[] args) {

```

```

public class P

```

```

static int
static int

```

```

public st
try {

```

```

catch

```

```

}

```

```

Boit

```

```

Syst
for

```

```

}
Syst
fo

```

```

}

```

```

System.out.p

```

```

3) public class ProdCons implements Thread {
    public static void main (String[] args) {

```

```

        try {
            Boite b = new Boite ();

```

```

            Cons c1 = new Cons (b);

```

```

                c2

```

```

                cn

```

Création de n Cons

```

            Prod p1 = new Prod (b, n, m);

```

```

                p2

```

```

                pn

```

Création de n Prod

```

            p.start ();

```

```

            sleep (100);

```

```

        }
        c.start ();

```

```

        catch (InterruptedException e) {
            System.out.println (e.getMessage());
        }

```

je hallucine!!!

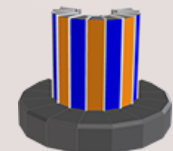
et les Ba
sur
per de

Prod envoient les messages via le run()

Cons se lancent une fois les messages envoyés

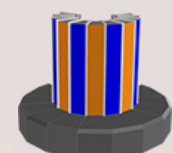
Un système producteur/consommateur

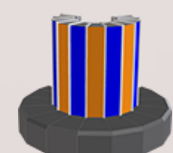
21

 La classe **Boite** est dotées des méthodes suivantes

```
public void envoyerMessage (String m)
```

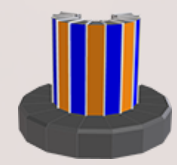
```
public String lireMessage ()
```

 La première est utilisée par les **Prod** et la seconde par les **Cons**

 Nous nous intéressons maintenant à la classe Prod qui écrit $2 \times \text{nbCons}$ messages dans la Boite (**nbCons** est le nombre de consommateurs récupéré dans la "classe principale"). Ces messages sont du format "message "+ **id** + «.»+**x**, **id** étant l'identité du producteur et **x** un compteur.

Un système producteur/consommateur, Q4

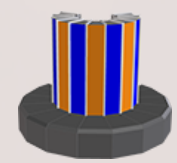
22



Écrivez les méthodes manquantes de la classe **Prod**
(le constructeur est déjà écrit).

Un système producteur/consommateur, Q4

22



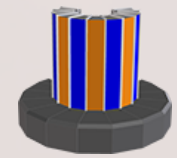
Écrivez les méthodes manquantes de la classe **Prod** (le constructeur est déjà écrit).

```
private void trace (String m) {
    System.out.println ("Prod " + monId + " : " + m);
}

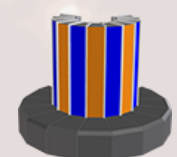
public void run() {
    this.trace ("he vais envoyer "+2*nbCons+" messages.");
    for (int i = 1; i <= 2*nbCons ; i++) {
        String msg = "message " + monId + "." + i;
        this.trace ("envoi du message " + msg);
        mi.envoyerMessage(msg);
    }
    this.trace ("au revoir");
}
```


Un système producteur/consommateur, Q5

23



Considérons désormais la classe **Cons** qui, lit le nombre de message transmis dans le constructeur et les affiche au fur et à mesure de leur arrivée avant de se terminer.



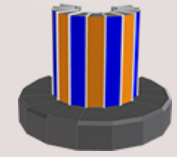
Écrivez les méthodes manquantes de la classe **Cons** (le constructeur est déjà écrit)

```
private void trace (String m) {
    System.out.println ("Cons " + monId + " : " + m);
}

public void run() {
    this.trace ("Je vais recevoir "+maxMsg+" messages");
    while (maxMsg > 0) {
        String msg = mi.lireMessage();
        this.trace ("lu " + msg);
        maxMsg--;
    }
    this.trace ("au revoir");
}
```


Un système producteur/consommateur

24



Nous considérons enfin la classe **Boite** qui n'est capable de stocker qu'un seul message. Ainsi, la primitive **envoyerMessage** doit-être bloquante si un message non consommé se situe dans l'objet. De même, la méthode **lireMessage** est bloquante si aucun message n'a encore été écrit, ou si le dernier message écrit a déjà été lu.

Un système producteur/consommateur, Q6

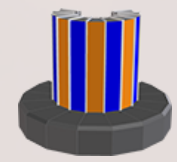
25

```
private String messageUnique = "";  
private boolean messagePresent = false;  
  
private final ReentrantLock l = new ReentrantLock();  
private final Condition ecrire = l.newCondition();  
private final Condition lire = l.newCondition();
```

- 🔊 Note : il n'y a pas besoin de constructeur spécifique ici, le constructeur par défaut de java (et l'initialisation des attributs) suffisant largement.
- 🔊 Il nous faut impérativement de quoi stocker un message. Un booléen est plus sur pour indiquer la présence d'un message même si on pourrait utiliser le cas où le message vaut "". Ensuite, il nous faut bien un **ReentrantLock** muni de deux conditions, l'une dédiée au blocage de l'écriture si le message existe et l'autre au blocage de la lecture si aucune message n'existe ou n'a été écrit depuis la dernière lecture.
- 🔊 Ici, un constructeur n'est pas nécessaire (on utilise le constructeur par défaut), car tous les attributs sont initialisés à leur déclaration (mais une solution avec constructeur est aussi correcte si les attributs ne sont pas initialisés à leur déclaration).
- 🔊 Notons que le booléen peut également être supprimé si on utilise la valeur null (dans ce cas, prévoir une initialisation correcte) ou "" de la chaîne contenant le message en lieu et place.

Un système producteur/consommateur, Q7

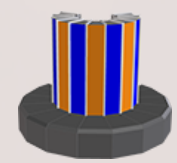
26



Écrivez le source de la méthode **envoyerMessage**

Un système producteur/consommateur, Q7

26

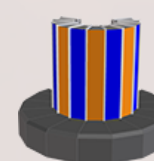


Écrivez le source de la méthode **envoyerMessage**

```
public void envoyerMessage (String m) {  
    try {  
        l.lock();  
        while (messagePresent) {  
            ecrire.await();  
        }  
        messagePresent = true;  
        messageUnique = m;  
        lire.signalAll();  
    }  
    catch (Exception e) {  
        e.printStackTrace();  
    }  
    finally {  
        l.unlock();  
    }  
}
```


Un système producteur/consommateur, Q7

26

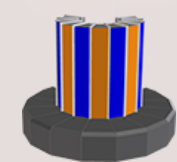
 Écrivez le source de la méthode **envoyerMessage**

Question 7

```
public void envoyerMessage (String m) {  
    l.lock  
    while (full) {  
        try {  
            lecture.await();  
        } catch (InterruptedException e) {  
            System.err.println("An error occurred");  
        }  
    }  
    full = true;  
    item = m;  
    lecture.signalAll();  
    l.unlock();  
}
```

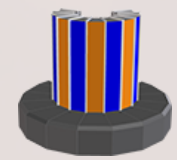

Un système producteur/consommateur, Q8

27

 Écrivez le source de la méthode **lireMessage**

Un système producteur/consommateur, Q8

27



Écrivez le source de la méthode **lireMessage**

```
public String lireMessage () {  
    try {  
        l.lock();  
        while (! messagePresent) {  
            lire.await();  
        }  
        messagePresent = false;  
        ecrire.signalAll();  
    }  
    catch (Exception e) {  
        e.printStackTrace();  
    }  
    finally {  
        l.unlock();  
    }  
    return messageUnique;  
}
```


Combien de temps pour avoir 20 points?

Combien de temps pour avoir 20 points?

28



Des questions?

29

